

OMNIX Newly Added Files - Full Contents

Bundle contains the exact full text of the four newly added repo files from the security-fix worktree.

No file contents are summarized or omitted. Long source lines are visually wrapped in the PDF only.

Manifest

services/github-app/tests/job_complete.test.ts

bytes: 1151

sha256: 7292d9e3dd5c82e9c12e251c67ffd26f31cc3d31ebae4d3114bafa23180b26c2

src/omnix/studio/security.py

bytes: 598

sha256: 9dcb8f86a75a3df78d72825f7e9f285a85901aa1b18e717e045ece6419f9b2a9

tests/studio/test_workspace_security.py

bytes: 697

sha256: 84dc19c4ade1993b5df078ebcbe46ca03bdb3098d3d93318d58dad9ecf1df5a

tests/test_agents_tools_security.py

bytes: 762

sha256: 3fb3020497fc0a03eec472beba3a0d8bd8b8992ac9ac1e9a195d14deb032cd86

1. services/github-app/tests/job_complete.test.ts

bytes=1151 sha256=7292d9e3dd5c82e9c12e251c67ffd26f31cc3d31ebae4d3114bafa23180b26c2

```
import {
  sanitizeReplicationBranchName,
  validateJobCompletePayload,
  validateTargetPath,
} from "../../src/handlers/job_complete.js";

describe("job-complete payload validation", () => {
  test("accepts normal repository paths", () => {
    expect(validateTargetPath("src/main/java/Foo.java")).toBe("src/main/java/Foo.java");
  });

  test("rejects path traversal and absolute paths", () => {
    expect(() => validateTargetPath("../github/workflows/pwn.yml")).toThrow();
    expect(() => validateTargetPath("/tmp/pwn")).toThrow();
    expect(() => validateTargetPath("src\\..\\pwn")).toThrow();
  });

  test("rejects workflow file writes", () => {
    expect(() => validateTargetPath(".github/workflows/omnix.yml")).toThrow();
  });

  test("rejects malformed repository names", () => {
    expect(() =>
      validateJobCompletePayload({
        job_id: "job-1",
        installation_id: 1,
        repo: "owner/repo/extra",
        units: [],
      })
    ).toThrow();
  });

  test("sanitizes branch suffixes", () => {
    expect(sanitizeReplicationBranchName("job/../../main")).toBe(
      "omnix/replicate/job-----main",
    );
  });
});
```

2. src/omnix/studio/security.py

bytes=598 sha256=9dcb8f86a75a3df78d72825f7e9f285a85901aa1b18e717e045ece6419f9b2a9

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
from fastapi import HTTPException
```

```
def safe_workspace_file_path(root: Path, raw_path: str) -> tuple[Path, str]:
    cleaned = (raw_path or "").replace("\\", "/")
    rel = Path(cleaned)
    if rel.is_absolute() or "." in rel.parts or not cleaned.strip("/"):
        raise HTTPException(400, "invalid path")
    resolved_root = root.resolve()
    full = (resolved_root / rel).resolve()
    if full != resolved_root and resolved_root not in full.parents:
        raise HTTPException(400, "invalid path")
    return full, rel.as_posix()
```

3. tests/studio/test_workspace_security.py

bytes=697 sha256=84dc19c4ade1993b5df078ebcbe46ca03bdb3098d3d933318d58dad9ecf1df5a

```
from __future__ import annotations

import pytest
from fastapi import HTTPException

from omnix.studio.security import safe_workspace_file_path

def test_workspace_file_path_rejects_parent_traversal(tmp_path):
    project = tmp_path / "project"
    project.mkdir()

    with pytest.raises(HTTPException):
        safe_workspace_file_path(project, "../escape.txt")
    assert not (tmp_path / "escape.txt").exists()

def test_workspace_file_path_accepts_project_relative_path(tmp_path):
    project = tmp_path / "project"
    project.mkdir()

    path, rel = safe_workspace_file_path(project, "src/app.py")

    assert path == (project / "src" / "app.py").resolve()
    assert rel == "src/app.py"
```

4. tests/test_agents_tools_security.py

bytes=762 sha256=3fb3020497fc0a03eec472beba3a0d8bd8b8992ac9ac1e9a195d14deb032cd86

```
from __future__ import annotations

from omnix.agents.tools import OmnixTools


def test_read_file_rejects_parent_traversal(tmp_path):
    project = tmp_path / "project"
    project.mkdir()
    outside = tmp_path / "secret.txt"
    outside.write_text("secret", encoding="utf-8")

    result = OmnixTools(str(project)).read_file("../secret.txt")

    assert "error" in result
    assert "outside project" in str(result["error"])


def test_read_file_accepts_project_relative_file(tmp_path):
    project = tmp_path / "project"
    project.mkdir()
    (project / "ok.py").write_text("print('ok')\n", encoding="utf-8")

    result = OmnixTools(str(project)).read_file("ok.py")

    assert result["file"] == "ok.py"
    assert "print('ok')" in str(result["content"])
```